ROUTLEDGE DIRECT | REFERENCE | OCTOBER 2025

Data dictionary

Every table and every column, raw and clean

The twenty six tables in BigQuery project ecomrl: what each column holds, how the tables join, which table answers which question, and the mistakes this data makes easy. Simulated data, not Taylor & Francis performance.

How to read this

Twenty six tables sit in BigQuery project `ecomrl`, in two datasets.

Layer	Tables	Built by	What they are for
`raw`	10	loaded as delivered	exactly what the source systems handed over
`clean`, fixing	6	`03_clean_layer.sql`	repairs everything the audit found
`clean`, reporting	4	`04_reporting_tables.sql`	the four tables the Power BI report reads
`clean`, customers	2	`05_customer_value.sql` and `01_customer_value.ipynb`	one row per person, and one row per acquisition channel
`clean`, reconciliation	4	`06_reconciliation_attribution.sql`	platform claims, journeys and affiliate transactions

The rule that makes this work. No value in `raw` is ever changed; the GA4 table is only reshaped once into the standard export structure, when section 0 of `01_load_and_checks.sql` is needed. Every fix happens once, in the clean layer, and is recorded in the data issues log. So it is always possible to show what the source said and what was done to it, which is the difference between an analysis someone can audit and one they have to take on trust.

Which layer to query. Day to day, only `clean`. Go back to `raw` for one reason: to show what the source said before anything was changed.

Types are the BigQuery types as the tables hold them: STRING, INT64, FLOAT64 (a decimal number), DATE, TIMESTAMP and BOOL (true or false). The GA4 export also has STRUCT and ARRAY columns, described with that table. **Row counts** are the counts in BigQuery.

Data windows. Orders, customers, refunds, spend, affiliates, `campaign_weekly`, `platform_weekly`, `affiliate_transactions` and the two customer tables cover 1 October 2022 to 30 September 2025. Refund dates run on to 9 October 2025, because a refund comes after its order. The GA4 export covers 1 October 2024 to 30 September 2025, so `sessions`, `funnel_daily`, `weekly_health`, `journey_touches` and `order_journeys` cover that year only. The reconciliation uses the 51 complete weeks of that year, the weeks starting 7 October 2024 to 22 September 2025.

Conventions used throughout

Convention	Meaning
`_usd`	US dollars, the reporting currency, already converted
`_local`	the original currency: what the customer paid in on orders, what the ad account was billed in on spend
`_ts`	a timestamp
`_date`	a calendar date, no time
`_key`	a value that has been lower cased or cleaned up so joins do not miss
`person_id`	a human being, after merging duplicate accounts
`customer_id`, `account_id`	a single login. One person can hold two
weeks	always start on Monday
mature customer	a person whose first order was on or before 30 September 2024, so they have had a full twelve months to come back (`has_full_12m`)

Revenue naming: read this before using any revenue column

Net revenue, in every report, chart and table, means revenue after discounts and after refunds, excluding tax and shipping charged. In the tables it is `net_revenue_after_refund_usd`.

Some columns do not say what they hold. Three tables (`raw.orders`, `clean.orders` and `clean.channel_daily`) have a column called `net_revenue_usd` that holds revenue **before** refunds, and `platform_weekly` has a column with the same name that holds revenue **after** refunds. Same name, different meaning. Before and after refunds differ by $388,385 across the whole business.

Table	Column	What it holds
raw.orders, clean.orders	net_revenue_usd	After discount, **before refunds**. Revenue before refunds
clean.orders	net_revenue_after_refund_usd	After discount and refunds. **Net revenue**
clean.channel_daily	net_revenue_usd	**Before refunds**, despite the name. Never shown on the report under this name
clean.channel_daily, clean.campaign_weekly	net_revenue_after_refund_usd	**Net revenue**
clean.platform_weekly	net_revenue_usd	**After** refunds, despite the name. **Net revenue**
clean.platform_weekly, clean.order_journeys, clean.affiliate_transactions	revenue_before_refunds_usd	Revenue before refunds, the basis the ad platforms claim on
clean.order_journeys, clean.journey_touches, clean.affiliate_transactions	net_revenue_after_refund_usd	**Net revenue**
clean.customer_value	revenue_usd, revenue_12m_usd, first_order_net_revenue_usd	**Net revenue**
raw.affiliates, clean.affiliate_transactions	order_value_usd, network_order_value_usd	The value the affiliate network claims against. Equal to the order's revenue before refunds on every transaction
clean.sessions, clean.funnel_daily	revenue_usd	What GA4 recorded on the purchase event: before refunds, and missing the orders GA4 never saw
clean.ad_spend, clean.channel_daily, clean.campaign_weekly, clean.platform_weekly	platform_revenue_usd	What the ad platform claims. Never refunded

Three rules follow from it.

1. **A column called `net_revenue_usd` that holds revenue before refunds is never shown on a report under that name.** Where revenue before refunds is shown at all, it is labelled "revenue before refunds".
2. **Before using a revenue column, check this table, not the column name.**
3. **Revenue before refunds is kept, not deleted**, so totals can be tied back to the order system. $12,296,260 before refunds, less $388,385 of refunds, is exactly $11,907,875 of net revenue.

The twenty six tables at a glance

Dataset	Table	One row is	Rows
raw	orders	an order, real or not	76,058
raw	order_items	a line on an order	134,284
raw	customers	an account	58,432
raw	refunds	a refund	2,390
raw	google_ads	a Google campaign on a day	9,864
raw	meta_ads	a Meta ad set on a day	19,728
raw	affiliates	an affiliate network transaction	17,369
raw	fx	a currency on a day	3,288
raw	promos	a sale	12
raw	events	a GA4 event	6,009,850
clean	channel_map	a source, medium and campaign combination	50
clean	customer_accounts	an account, pointing at its person	58,419
clean	customers	a person	58,101
clean	orders	a real order	75,918
clean	ad_spend	a campaign or ad set on a day, both platforms	29,592
clean	sessions	a website session	1,264,269
clean	channel_daily	a day, channel and market	34,356
clean	funnel_daily	a day, channel, market and device	39,721
clean	campaign_weekly	a week, market, channel and paid campaign	2,370
clean	weekly_health	a week of the GA4 year	53
clean	customer_value	a person, with first order, value and RFM	58,101
clean	unit_economics	an acquisition channel	13
clean	platform_weekly	a week and an ad platform	316
clean	journey_touches	a session in the 30 days before an order GA4 saw	51,375
clean	order_journeys	an order of the GA4 year	31,692
clean	affiliate_transactions	an affiliate transaction on a real order	17,344

raw: the ten tables as they arrived

Loaded as delivered; no value in them is ever changed. `01_load_and_checks.sql` checks they are complete and join up, and `02_data_audit.sql` sizes every problem found in them.

raw.orders

One row per order, including staff test orders. Source: the commerce platform. 76,058 rows.

Column	Type	Meaning
order_id	STRING	Order reference. Unique
customer_id	STRING	The account that ordered. Joins to `customers.customer_id`
order_ts_utc	TIMESTAMP	When the order was placed, in UTC
market	STRING	US, UK or ROW, the storefront used. Always the account's market
local_currency	STRING	What the customer paid in: USD in the US, GBP in the UK, EUR in ROW, on every order
gross_revenue_local	FLOAT64	Order value before discount, in the customer's currency
net_revenue_local	FLOAT64	Order value after discount, in the customer's currency
gross_revenue_usd	FLOAT64	Order value before discount, in dollars
discount_usd	FLOAT64	Sale discount. Zero when no code was used
net_revenue_usd	FLOAT64	Order value after discount, in dollars, **before refunds**. Not net revenue; see revenue naming
shipping_charged_usd	FLOAT64	What the customer paid for delivery. Not revenue
tax_usd	FLOAT64	Tax collected. **Not revenue; excluded from every figure**
product_cost_usd	FLOAT64	What the books on the order cost the business
shipping_cost_usd	FLOAT64	What delivery cost the business, which is not what the customer paid
payment_fee_usd	FLOAT64	Card and payment processing fee
promo_code	STRING	The sale code used, or null. Joins to `promos.promo_code`. Null on 74.6 percent of orders, because most orders fall outside a sale
item_count	INT64	Lines on the order. Always equal to the lines in `order_items`
device	STRING	desktop, mobile or tablet
utm_source	STRING	Where the visit came from, as tagged. **Written several ways**; see `clean.channel_map`
utm_medium	STRING	The kind of traffic, as tagged. Also written several ways
utm_campaign	STRING	Campaign as tagged. Null on 31.6 percent of orders, mostly organic search, direct and referral visits, which carry no campaign

Column	Type	Meaning
fx_rate_used	FLOAT64	The rate used to convert the order into dollars. Equal to `raw.fx` for that day and currency on every order

Orders run from $0.01 to $1,008.60. The 140 orders under $15 belong to 13 staff and QA accounts on `tandf-test.com`, $634.41 between them, and none of them has order lines. They are removed in the clean layer.

raw.order_items

One row per line on an order. Source: the commerce platform. 134,284 rows.

Column	Type	Meaning
order_id	STRING	Joins to `orders.order_id`
product_id	STRING	The book
quantity	INT64	Copies of the title on this line
unit_price_usd	FLOAT64	Price per copy before discount
discount_usd	FLOAT64	Discount on this line
line_total_usd	FLOAT64	Line value after discount
unit_cost_usd	FLOAT64	Cost per copy to the business

The lines cover all 75,918 real orders; the 140 staff orders have none. **Lines and order totals are a cent or two apart on 2,512 orders**, every one of them discounted: the discount was rounded line by line. Lines come to $4.00 less than the orders overall, and no order is more than two cents out. **Order totals are the source for revenue.** Lines are used only for what was bought.

raw.customers

One row per account, not per person. Source: the commerce platform. 58,432 rows.

Column	Type	Meaning
customer_id	STRING	Account reference. Unique
email	STRING	The address on the account. **Not unique to a person**: some people hold two accounts
signup_date	TIMESTAMP	When the account was opened. No order comes before it
market	STRING	US, UK or ROW
country	STRING	Country name
marketing_opt_in	BOOL	Whether this account agreed to marketing email

The 13 accounts on `tandf-test.com` are staff and QA accounts. They are removed in the clean layer and never count as customers.

raw.refunds

One row per refund. Source: the commerce platform. 2,390 rows.

Column	Type	Meaning
refund_id	STRING	Refund reference. Unique
order_id	STRING	Joins to `orders.order_id`. One refund per order at most
refund_date	TIMESTAMP	When the refund was processed
refund_amount_usd	FLOAT64	Amount refunded. Never more than the order
refund_reason	STRING	Why it was refunded

2,390 refunds on 2,390 orders, $388,384.75 in all, 3.15 percent of real orders. 1,961 refund the whole order and 429 part of it. They come 4 to 44 days after the order, about 24 days on average, so 32 are dated after 30 September 2025 and the newest orders have not had all their refunds yet. By reason: damaged in transit 581 ($95,082.70), no longer needed 519 ($84,498.20), ordered by mistake 515 ($84,022.01), wrong edition 418 ($65,248.52), late delivery 357 ($59,533.32).

raw.google_ads

One row per campaign per day, as Google reports it. 9,864 rows: 9 campaigns on every one of 1,096 days.

Column	Type	Meaning
report_date	DATE	The day, on UTC
account	STRING	The ad account
campaign_id	STRING	Google's campaign id
campaign_name	STRING	Campaign name, the same string the clicks are tagged with in `utm_campaign`
campaign_type	STRING	The kind of campaign
market	STRING	US, UK or ROW
currency	STRING	What the account is billed in: GBP for the UK, USD for the US and ROW
cost	FLOAT64	Spend in the billing currency, **not dollars**
impressions	INT64	Times an ad was shown
clicks	INT64	Clicks
conversions	FLOAT64	Orders Google claims. Fractional
conversion_value	FLOAT64	Revenue Google claims, in the billing currency
reporting_timezone	STRING	UTC on every row

What a platform claims is not what happened. Google counts a conversion when it believes its ad contributed, which is a different question from what the order system recorded. Both are kept, and the reconciliation walks from one to the other.

raw.meta_ads

One row per ad set per day, as Meta reports it. 19,728 rows: 18 ad sets on every one of 1,096 days.

Column	Type	Meaning
report_date	DATE	The day, on Pacific time
account	STRING	The ad account
campaign_name	STRING	Campaign name
adset_name	STRING	Ad set within the campaign
objective	STRING	What the ad set was optimised for
market	STRING	US, UK or ROW
currency	STRING	Billing currency: USD for the US, GBP for the UK, EUR for ROW
spend	FLOAT64	Spend in the billing currency, **not dollars**
impressions	INT64	Times an ad was shown
clicks	INT64	Clicks
purchases_7d_click	FLOAT64	Purchases within seven days of a click
purchases_1d_view	FLOAT64	Purchases within one day by someone who **only saw** the ad
purchase_value_7d_click	FLOAT64	Revenue from the click purchases, in the billing currency. Meta puts no value on view purchases
reporting_timezone	STRING	America/Los_Angeles on every row

Over three years Meta reported 17,849.1 purchases after a click and 5,608.2 after only a view. Added together they flatter the channel, so `clean.platform_weekly` keeps them apart. Meta's day starts seven or eight hours after Google's and the order system's, so the two platforms are compared by week, never by day.

raw.affiliates

One row per affiliate network transaction. 17,369 rows.

Column	Type	Meaning
transaction_id	STRING	Network reference. Unique
partner_id	STRING	The partner the network paid
partner_type	STRING	coupon or content. **The most useful column in the table**
click_ts	TIMESTAMP	When the partner click happened
order_id	STRING	Joins to `orders.order_id`. **One transaction per order**: 17,369 transactions on 17,369 orders
order_value_usd	FLOAT64	The value the network claims against. Equal to the order's revenue before refunds on every transaction
commission_usd	FLOAT64	Commission payable: 8.0 percent of order value for coupon partners, 10.0 for content. **A real cost, taken out of gross profit**
status	STRING	approved, pending or reversed. Reversed means the network clawed it back, so it is not a cost. Approved and pending both are; pending is still owed

Coupon and content partners are different businesses under one label. A content partner writes something that sends a reader. A coupon site, the data suggests, is often found by someone already part way through a purchase. Coupon clicks land a median 435 seconds (7.25 minutes) before the order and content clicks 7,380 seconds (123 minutes); every click is between 20 seconds and just under 4 hours before its order.

25 transactions sit on staff test orders: $6.88 still standing and $1.00 reversed. They drop out of every clean table. **Partner ids do not match the source tags on orders**: voucherhub, student-codes and phd-toolkit never appear as a source, coupons-site appears as a source but is not a partner, and each partner's orders split about evenly between the two sources of its type. Partner type always matches the channel, so coupon and content can be compared, but single partners cannot be tied to their tags.

raw.fx

One row per currency per day. Source: finance, published rates. 3,288 rows, 3 currencies on every one of 1,096 days.

Column	Type	Meaning
rate_date	DATE	The day the rate applies to
currency	STRING	USD, GBP or EUR
rate_to_usd	FLOAT64	Multiply by this to get dollars. USD is always 1. GBP runs from 1.21999 to 1.32991 and EUR from 1.00337 to 1.06350

Converting at the daily rate rather than one average for the year is what makes spend and revenue comparable day by day.

raw.promos

One row per sale. 12 rows: four sales, run every year on the same dates.

Column	Type	Meaning
promo_id	STRING	Sale reference
promo_name	STRING	black_friday, new_year, mid_year or back_to_college
promo_code	STRING	The code customers typed. Joins to `orders.promo_code`
start_date	DATE	First day of the sale
end_date	DATE	Last day of the sale, inclusive
discount_pct	INT64	The discount
markets	STRING	Markets it ran in
applies_to	STRING	Which products it applied to

Sale	Dates each year	Discount
black_friday	20 November to 2 December	30 percent
new_year	3 to 18 January	15 percent
mid_year	20 May to 10 June	15 percent
back_to_college	8 August to 24 September	20 percent

No two sales overlap, every code was used only inside its own sale, and every coded order got exactly the sale's discount.

raw.events

One row per GA4 event. Source: the GA4 BigQuery export, loaded from four quarterly files. 6,009,850 rows on 363 days, 1 October 2024 to 30 September 2025.

This table is nested. Four columns hold a structure (STRUCT) of named values and two hold a list (ARRAY).

Column	Type	Meaning
event_date	STRING	The day as `YYYYMMDD`. **Text, not a date**: read it with `parse_date('%Y%m%d', event_date)`
event_timestamp	INT64	Microseconds since 1970. Read it with `timestamp_micros()`
event_name	STRING	Which event; see below
user_pseudo_id	STRING	The browser. The nearest thing to a visitor id
user_id	STRING	The logged in account, when there is one. 389,167 events carry one
device	STRUCT	`device.category` (desktop, mobile or tablet) and `device.operating_system`
geo	STRUCT	`geo.country`, the country name
traffic_source	STRUCT	`traffic_source.source`, `.medium` and `.name`. The campaign is in `.name`
event_params	ARRAY	Key and value pairs; see below
items	ARRAY	The books on the event: `item_id`, `item_name`, `price_in_usd`, `quantity`
ecommerce	STRUCT	`ecommerce.transaction_id` (the order id) and `ecommerce.purchase_revenue_in_usd`, filled on purchase events

The six events

event_name	Fires when	Events
session_start	A visit begins	1,264,269
page_view	Any page loads	3,160,670
view_item	A book page loads	1,294,696
add_to_cart	Add to basket is clicked	167,534
begin_checkout	The checkout page loads	94,217
purchase	The order is confirmed	28,464

Nothing is recorded between `begin_checkout` and `purchase`, so GA4 cannot show which checkout step a visitor left on. The tracking plan adds `add_shipping_info` and `add_payment_info` for that reason.

The event_params keys

Each value sits in a structure with three slots, and only one is filled. Ask for the wrong slot and the answer is null.

key	Value slot	What it is
ga_session_id	`int_value`	The session number, **unique only within one browser**
page_location	`string_value`	The page address
item_id	`string_value`	The book
value	`double_value`	The money amount, on checkout and purchase events
transaction_id	`string_value`	The order, joining to `orders.order_id`

PULLING ONE PARAMETER OUT

To put the session number on each row, reach into the list and keep the one key wanted:

```
(select p.value.int_value from unnest(event_params) as p where p.key =
'ga_session_id')
```

Read it inside out. `unnest(event_params)` turns the list into rows that can be filtered, `where p.key = 'ga_session_id'` keeps the one wanted, and `p.value.int_value` takes the slot that is filled. Ask for `string_value` instead and the answer is null, because a session number is a number.

`clean.sessions` has already done this for every session, so later work rarely needs it.

Four known problems in this table

Two days are missing: nothing was recorded on 13 and 14 May 2025, about 30,000 events judging by the days either side.

Some purchases were sent twice. 28,464 purchase events carry 27,947 order ids: 517 orders were sent twice, 324 in January 2025 and 193 in February. Count distinct `ecommerce.transaction_id`, never purchase events.

GA4 sees fewer orders than the order system: it misses 10.5 to 12.1 percent of orders every month, 17.2 in May 2025 because of the missing days. Visitors who decline tracking are not recorded, which is correct behaviour and the usual reason for a gap like this, though the data does not show the cause. The order system is used for totals and GA4 for shares and rates.

Event times inside a session are on a fixed schedule. Every event sits a fixed number of seconds after its session started (`begin_checkout` always 210, `purchase` 260), and the purchase event is stamped before the order system's time on 26,902 of 27,947 orders. The times show which session an event belongs to and in what order things happened, but not how long anything took. Session start times are sound.

clean: the six tables that fix the data

Built by `03_clean_layer.sql`.

clean.channel_map

One row per source, medium and campaign combination found in orders or in GA4, with the channel it belongs to. 50 rows, 13 channels, none unassigned.

Column	Type	Meaning
source_key	STRING	`utm_source` lower cased
medium_key	STRING	`utm_medium` lower cased
campaign_key	STRING	`utm_campaign` lower cased, with a missing campaign written `(none)`
channel	STRING	The channel. One of 13
channel_group	STRING	paid, affiliate, owned or organic
is_brand_search	BOOL	People searching for the shop by name
ad_platform	STRING	google or meta for paid search and paid social, tagged or not; null for everything else

Why a table and not a CASE in every query. The same rules have to classify orders, GA4 sessions and ad spend identically, or the three will never reconcile. Written once as a table, the rules can be read, and a new tag shows up as `unassigned` instead of quietly landing in the wrong channel. A missing campaign becomes `(none)` because a join on a column that can be null drops rows without saying so.

What the 50 combinations are. 14 sources, 9 mediums and 19 campaign values (18 campaigns plus `(none)`), all lower cased. Every combination appears both on raw orders and in GA4 sessions. On raw orders the tags as written give 20 source and medium pairs; lower cased, they give 17.

Channel	Group	Combinations	Made of
google_brand	paid	3	google, cpc, one brand campaign per market
google_nonbrand	paid	3	google, cpc, one non brand campaign per market
google_shopping	paid	6	google with cpc and with shopping, three shopping campaigns each
google_paid_untagged	paid	2	google with cpc and with shopping, no campaign
meta_prospecting	paid	9	facebook and fb with paid_social, meta_ads with paid-social, three prospecting campaigns each
meta_retargeting	paid	6	facebook and meta_ads with paid_social, three retargeting campaigns each
meta_untagged	paid	4	facebook and fb with paid_social, meta_ads with paid-social and with paid_social, no campaign
affiliate_coupon	affiliate	4	coupons-site and dealfinder, affiliate, with coupon_partners or no campaign
affiliate_content	affiliate	4	studyguide-blog with affiliate and academic-reads with referral, with content_partners or no campaign
email	owned	4	klaviyo and newsletter, email, with crm_weekly or no campaign
organic_search	organic	2	google and bing, organic
referral	organic	2	researchgate.net and tandfonline.com, referral
direct	organic	1	(direct), (none)

clean.customer_accounts

One row per account, pointing at the person who owns it. 58,419 rows: the 58,432 raw accounts less the 13 staff and QA accounts. The record of the deduplication.

Column	Type	Meaning
customer_id	STRING	The account
email_key	STRING	The cleaned up address that identifies the person
person_id	STRING	The person: the lowest account id among the accounts sharing an `email_key`
email	STRING	The address as the source had it
market	STRING	US, UK or ROW
country	STRING	Country name
marketing_opt_in	BOOL	As recorded on this account
signup_date	TIMESTAMP	When this account was opened

The matching rule. Lower case the address, remove any plus tag (`name+books@` becomes `name@`) on every domain, and remove dots only on gmail.com. Gmail ignores dots, so two Gmail addresses that differ only by dots are one inbox. Outlook, Yahoo, Proton and the university domains treat a dot as part of the address, so a match that ignores dots there is not proof of one person.

The rule merges 318 pairs: the 58,419 accounts belong to 58,101 people, and nobody holds more than two accounts. The 318 people placed 1,068 orders worth $207,386 before refunds. Across all 58,432 raw accounts the same rule finds 319, because one pair of staff accounts also matches. Removing dots on every domain would merge 101 more pairs (58,000 people); the repeat rate would be 19.59 percent instead of 19.62, 19.6 either way.

Nothing queries this table day to day. It is kept because it proves which accounts were merged and turns any account id back into a person. `journey_touches` also uses it to link the browsers a person logged in on.

clean.customers

One row per person. 58,101 rows.

Column	Type	Meaning
person_id	STRING	The person. **The customer key for the whole project**
email_key	STRING	The cleaned up address
accounts_held	INT64	1 for 57,783 people, 2 for 318
first_signup_date	TIMESTAMP	When they first opened an account
market	STRING	From their earliest account
country	STRING	From their earliest account
marketing_opt_in	BOOL	True if **either** account opted in. 37,183 people, 64.0 percent

clean.orders

One row per real order, with the person, the channel, refunds and profit attached. 75,918 rows: the 76,058 raw orders less the 140 staff test orders, which drop out through the join to `customer_accounts`.

Column	Type	Meaning
order_id	STRING	Order reference
person_id	STRING	The person who ordered
account_id	STRING	The account used, the raw `customer_id`
order_ts_utc	TIMESTAMP	When it was placed
order_date	DATE	The day
order_week	DATE	Monday of that week
order_month	DATE	First of that month
market	STRING	US, UK or ROW
channel	STRING	From `channel_map`: the channel recorded on the order, so last click
channel_group	STRING	paid, affiliate, owned or organic
is_brand_search	BOOL	Brand search
ad_platform	STRING	google or meta on every paid order, tagged or untagged; null otherwise
campaign	STRING	Campaign as tagged, original spelling
device	STRING	desktop, mobile or tablet
gross_revenue_usd	FLOAT64	Before discount
discount_usd	FLOAT64	Discount given
net_revenue_usd	FLOAT64	After discount, **before refunds**. See revenue naming
refunded_usd	FLOAT64	Refunded against this order. Zero when nothing was
net_revenue_after_refund_usd	FLOAT64	After discount and refunds. **Net revenue**
product_cost_usd	FLOAT64	Cost of the books
shipping_cost_usd	FLOAT64	Cost to deliver
payment_fee_usd	FLOAT64	Payment processing
affiliate_commission_usd	FLOAT64	Commission still standing on this order. Zero when there is none
gross_profit_usd	FLOAT64	**The profit figure.** Formula below
promo_code	STRING	Sale code used, or null
used_promo	BOOL	A sale code was used
item_count	INT64	Lines on the order

```
gross_profit_usd =
    net_revenue_usd           -- before refunds
  - refunded_usd
  - product_cost_usd
  - shipping_cost_usd
  - payment_fee_usd
  - affiliate_commission_usd
```

Tax is not revenue and never enters. Ad spend is not taken off here, because spend belongs to a channel and a day, not to an order.

Totals: revenue before refunds $12,296,260.18, refunds $388,384.75, net revenue $11,907,875.43, affiliate commission $227,336.14, gross profit $7,421,060.08, a 62.3 percent margin on net revenue. Raw orders come to $12,296,894.59; the $634.41 difference is the staff orders. The network shows $227,343.02 of commission still owed; the $6.88 difference is on 24 transactions on those staff orders.

clean.ad_spend

One row per Google campaign or Meta ad set per day, both platforms in one table, converted to dollars at that day's rate. 29,592 rows: 9,864 from Google and 19,728 from Meta.

Column	Type	Meaning
platform	STRING	google or meta
report_date	DATE	The day, in the platform's own time zone
report_week	DATE	Monday of that week
market	STRING	US, UK or ROW
campaign_name	STRING	As the ad account writes it
channel	STRING	From `channel_map`, so spend and orders are classified alike
billing_currency	STRING	What the account was billed in
spend_local	FLOAT64	Spend as billed
rate_to_usd	FLOAT64	The day's rate
spend_usd	FLOAT64	**Spend in dollars, rounded to the cent. Use this one**
impressions	INT64	Times shown
clicks	INT64	Clicks
platform_conversions	FLOAT64	Orders the platform claims. **For Meta, click and view purchases added together**
platform_revenue_usd	FLOAT64	Revenue the platform claims, converted. For Meta, click purchases only
reporting_timezone	STRING	UTC for Google, America/Los_Angeles for Meta

Spend: Google $609,509.99, Meta $637,778.95, $1,247,288.94 in all, every row with a channel. Google claims 29,086.2 conversions and Meta 23,457.3 purchases, clicks and views together.

Platform	Billing currency	Rows	Spend as billed	Spend in dollars
google	GBP	3,288	119,320.01	$153,581.01
google	USD	6,576	455,928.98	$455,928.98
meta	EUR	6,576	126,672.94	$130,343.87
meta	GBP	6,576	122,858.63	$158,177.13
meta	USD	6,576	349,257.95	$349,257.95

UK spend is billed in pounds: 242,178.64 GBP is $311,758.14, so the dollar figure is 28.7 percent higher than the number in the export. Read as dollars without converting, the UK would look cheaper to run than it is.

clean.sessions

One row per website session, with the GA4 nesting gone. 1,264,269 rows on 363 days, **1 October 2024 onwards only**.

Column	Type	Meaning
session_key	STRING	Browser and session number joined. The unique session id
user_pseudo_id	STRING	The browser
user_id	STRING	The logged in account, when there is one
session_date	DATE	The day the session started
session_week	DATE	Monday of that week
session_start_ts	TIMESTAMP	When it started
channel	STRING	From `channel_map`
channel_group	STRING	paid, affiliate, owned or organic
is_brand_search	BOOL	Brand search
source	STRING	Source, lower cased
medium	STRING	Medium, lower cased
campaign	STRING	Campaign, lower cased, or null where there was none
device	STRING	desktop, mobile or tablet
country	STRING	Country name
market	STRING	US, UK or ROW, worked out from the country
events	INT64	Events in the session
page_views	INT64	Page views
item_views	INT64	Book page views
add_to_carts	INT64	Add to cart events
checkouts	INT64	Checkout events
converted	BOOL	The session ended in a purchase

Column	Type	Meaning
order_id	STRING	The order, taken once per session, so a purchase sent twice counts once
revenue_usd	FLOAT64	Revenue on that purchase event, as GA4 recorded it. Before refunds

1,264,269 sessions from 1,233,851 browsers. 852,687 viewed a book, 167,534 added to cart, 94,217 reached checkout and 27,947 bought, carrying 27,947 different order ids. 42 of those orders are staff test orders that are not in `clean.orders`.

Why the session key is two things joined. A GA4 session number is unique only within one browser, so two browsers can both have session 1. **Why traffic source is taken with `min()` and not grouped on.** It is recorded on every event but belongs to the session; grouping on it would split a session in two if one event disagreed.

clean: the four reporting tables

Built by `04_reporting_tables.sql`. The Power BI report reads these four and nothing else.

clean.channel_daily

One row per day, channel and market. **The main table the report reads.** 34,356 rows.

Column	Type	Meaning
date	DATE	The day
week	DATE	Monday of that week
month	DATE	First of that month
channel	STRING	The channel
market	STRING	US, UK or ROW
spend_usd	FLOAT64	Ad spend
impressions	INT64	Times shown
clicks	INT64	Clicks
platform_conversions	FLOAT64	Orders the platform claims
platform_revenue_usd	FLOAT64	Revenue the platform claims
orders	INT64	Orders the business recorded, by last click
net_revenue_usd	FLOAT64	Revenue **before refunds**, despite the name
refunded_usd	FLOAT64	Refunds
net_revenue_after_refund_usd	FLOAT64	**Net revenue**. The report's revenue column
gross_profit_usd	FLOAT64	Gross profit
affiliate_commission_usd	FLOAT64	Commission on the orders credited to this channel. It travels with the order, so it sits on whichever channel last click credits, not only on the two affiliate channels
discount_usd	FLOAT64	Discounts given
orders_with_promo	INT64	Orders that used a code
new_customers	INT64	People whose first order was on this day, channel and market
sessions	INT64	Sessions. **Null, not zero, before October 2024**, and on any day GA4 recorded nothing for that channel and market, such as 13 and 14 May 2025
converting_sessions	INT64	Sessions that bought. Null wherever `sessions` is null

Totals: 75,918 orders, $12,296,260.18 before refunds, $11,907,875.43 net revenue, $7,421,060.08 gross profit and $227,336.14 commission, all equal to `clean.orders`; $1,247,288.94 of spend; 58,101 new customers, one per person; 1,264,269 sessions. Over the GA4 year alone it holds 31,692 orders, the same

as `order_journeys`.

> **WHY FULL OUTER JOINS**
>
> Spend, orders and sessions are three sources, and each can exist without the others. A channel can spend on a day it sells nothing, and organic search sells with no spend at all. An inner join would keep only days where everything existed, and quietly drop the costly days that produced nothing.
>
> So they are full outer joins, and a zero is a real zero. Sessions before October 2024 stay null, because the GA4 export does not reach back that far, and so do sessions on any day GA4 recorded nothing for that channel and market, such as 13 and 14 May 2025. Null means not known; zero would claim nobody visited.

The paid channels on last click:

Channel	Spend	New customers	Cost per new customer	Internal ROAS	Profit return on spend
meta_prospecting	$512,242	9,750	$52.54	2.61	0.69
google_nonbrand	$331,980	9,514	$34.89	5.30	2.31
google_shopping	$200,064	5,577	$35.87	4.19	1.65
meta_retargeting	$125,537	3,150	$39.85	5.54	2.53
google_brand	$77,465	4,968	$15.59	14.28	7.98

The other eight channels have no spend, so those three columns are blank, not zero.

clean.funnel_daily

One row per day, channel, market and device, from `clean.sessions`. 39,721 rows on 363 days.

Column	Type	Meaning
date	DATE	The day
week	DATE	Monday of that week
channel	STRING	The channel
channel_group	STRING	paid, affiliate, owned or organic
market	STRING	US, UK or ROW
device	STRING	desktop, mobile or tablet
sessions	INT64	Sessions
sessions_viewed_item	INT64	Sessions that viewed a book
sessions_added_to_cart	INT64	Sessions that added to cart
sessions_reached_checkout	INT64	Sessions that reached checkout
sessions_purchased	INT64	Sessions that bought
revenue_usd	FLOAT64	Revenue GA4 recorded on those purchases. Before refunds
conversion_rate_pct	FLOAT64	`sessions_purchased` over `sessions`, as a percentage, for reading one row. **Never average it across rows**

1,264,269 sessions and 27,947 purchases, equal to `clean.sessions`. Each step counts **sessions that reached it**, not events. Over the GA4 year 67.4 percent of sessions viewed a book, 19.6 percent of those added to cart, 56.2 percent of those reached checkout and 29.7 percent of those bought: 2.21 percent of sessions ended in a purchase.

clean.campaign_weekly

One row per week, market, channel and paid campaign, for the 15 campaigns in the five tagged paid channels. 2,370 rows, no duplicates. The report's campaign page reads it.

Column	Type	Meaning
week	DATE	Monday of the week
market	STRING	US, UK or ROW
channel	STRING	One of the five tagged paid channels
platform	STRING	google or meta
campaign	STRING	Campaign name, **lower cased**, so the six Meta campaigns that arrive in two spellings are not split
spend_usd	FLOAT64	Ad spend. Zero, not null, in a week with orders but no spend
impressions	INT64	Times shown
clicks	INT64	Clicks
platform_conversions	FLOAT64	Orders the platform claims. For Meta, clicks and views
platform_revenue_usd	FLOAT64	Revenue the platform claims
orders	INT64	Orders credited to this campaign by last click
new_customers	INT64	Of those, the person's first ever order
net_revenue_after_refund_usd	FLOAT64	**Net revenue**
gross_profit_usd	FLOAT64	Gross profit

Weekly, not daily, because Google reports on UTC and Meta on Pacific time; over a week the difference washes out. **Paid campaigns only**, because CTR, cost per click and cost per order need spend. Spend is $1,247,288.94, all of it. Orders, net revenue and new customers equal the tagged paid orders: 37,461 orders, $5,735,882.61 and 32,959 new customers. The 775 paid orders that lost their campaign tag are not here (trap 8).

clean.weekly_health

One row per week of the GA4 year: 53 rows, the weeks of 30 September 2024 to 29 September 2025. It flags any week where the tracking or the conversion rate moves outside its normal range, without being told which weeks to look at.

Column	Type	Meaning
week	DATE	Monday of the week
days_in_window	INT64	Days of the week inside the GA4 year. 7, except the two part weeks at either end
days_with_data	INT64	Days with GA4 data, found from a calendar, because a missing day has no rows anywhere
sessions	INT64	Sessions
purchasing_sessions	INT64	Sessions that bought

Column	Type	Meaning
ga4_orders	INT64	Orders GA4 saw that exist in the order system, so staff test orders are left out
system_orders	INT64	Orders the order system recorded
purchase_events	INT64	Purchase events, repeats included
purchased_orders	INT64	Distinct order ids on those events
conversion_rate	FLOAT64	`purchasing_sessions / sessions`
conversion_rate_normal	FLOAT64	Median over complete weeks: 0.02204
capture_rate	FLOAT64	`ga4_orders / system_orders`, the share of real orders GA4 saw
capture_rate_normal	FLOAT64	Median over complete weeks: 0.8847
events_per_order	FLOAT64	`purchase_events / purchased_orders`. 1.00 in a healthy week
events_per_order_normal	FLOAT64	Median: 1.0
flag_missing_days	BOOL	Fewer days with data than days in the window
flag_capture_rate	BOOL	Capture rate more than 10 percent from normal, either way
flag_duplicate_purchases	BOOL	Events per order more than 5 percent above normal
flag_conversion_rate	BOOL	Conversion rate more than 20 percent from normal, either way

Ratios, not volumes, because sessions rise every term start and a volume check would flag every September. **Medians, not averages**, because broken weeks would pull an average towards themselves.

Week	Flagged for	Value
20 January 2025	duplicate purchases	1.34 events per order
27 January 2025	duplicate purchases	1.37
3 February 2025	duplicate purchases	1.31
12 May 2025	missing days, capture rate	5 of 7 days, capture 0.611
19 May 2025	conversion rate	2.790 percent
9 June 2025	conversion rate	2.882 percent
16 June 2025	conversion rate	2.804 percent
23 June 2025	conversion rate	2.705 percent
30 June 2025	conversion rate	2.695 percent

In Power BI each week also gets a health type: 44 OK, 4 data issue and 5 performance. In the five conversion weeks every day has data, capture is normal and each order fires once, so the tracking looks healthy. The mid-year sale does not explain them: its two full weeks, 26 May and 2 June, were not flagged (2.435 and 2.385 percent), and 16, 23 and 30 June had no sale days. The cause is open.

clean: the two customer tables

clean.customer_value

One row per person: who they are, how they arrived, what they are worth and where they are now. Built by `05_customer_value.sql`. 58,101 rows, 36 columns.

Column	Type	Meaning
person_id	STRING	The person
first_order_date	DATE	Day of their first order
cohort_month	DATE	First of that month
acquisition_channel	STRING	Channel of the **first** order
acquisition_channel_group	STRING	paid, affiliate, owned or organic
market	STRING	Market of the first order
first_order_device	STRING	Device of the first order
acquired_on_promo	BOOL	The first order used a sale code
orders	INT64	Orders ever
orders_12m	INT64	Orders in the 365 days from the first order
orders_with_promo	INT64	Orders that used a code
last_order_date	DATE	Most recent order
revenue_usd	FLOAT64	Net revenue ever, after refunds
gross_profit_usd	FLOAT64	Gross profit ever
revenue_12m_usd	FLOAT64	Net revenue in the 365 days from the first order
gross_profit_12m_usd	FLOAT64	Gross profit in the same window. **Twelve month value**
is_repeat_customer	BOOL	More than one order ever
repeated_within_12m	BOOL	More than one order in the 365 days from the first
has_full_12m	BOOL	First order on or before 30 September 2024. **Read this before averaging anything with 12m in its name**
marketing_opt_in	BOOL	From `clean.customers`
accounts_held	INT64	Accounts merged into this person
first_order_id	STRING	The first order: the earliest, with `order_id` breaking a tie on the same time
first_order_in_sale_window	BOOL	The first order fell on a sale day
first_order_sale_name	STRING	Which sale, or null
first_order_price_group	STRING	sale, used the code; sale period, full price; or outside any sale
first_order_net_revenue_usd	FLOAT64	First order net revenue

Column	Type	Meaning
first_order_gross_profit_usd	FLOAT64	First order gross profit
first_order_discount_usd	FLOAT64	Discount on the first order
first_order_commission_usd	FLOAT64	Commission on the first order. For the two affiliate channels this is what the customer cost to win
recency_days	INT64	Days from the last order to 30 September 2025
r_score	INT64	Recency score 1 to 5, fifths with ties kept together. 5 is the most recent
f_score	INT64	Frequency score on fixed bands: 1 order, 2, 3, 4 or 5, 6 or more
m_score	INT64	Monetary score 1 to 5 on lifetime gross profit, fifths with ties kept together
rfm_score	STRING	The three scores as text, for example 515
rfm_group	STRING	One of ten groups from R and F
lifecycle_stage	STRING	New, Active repeat, Cooling or Lapsed

Totals: 58,101 people, 75,918 orders, $11,907,875.43 of net revenue and $7,421,060.08 of gross profit, equal to `clean.orders`. Every first order is the person's earliest, and new customers by acquisition channel equal `channel_daily` on all 13 channels.

Repeat rate is 19.6 percent: 7,175 of the 36,569 mature customers ordered again within twelve months of their first order. 13,035 people (22.4 percent of all 58,101) have ever ordered twice; that is **lifetime repeat across all customers**, a different measure with no fixed window, and is only quoted with that label.

> **WHY HAS_FULL_12M EXISTS**
>
> Twelve month value is measured from each person's own first order. Someone whose first order was in August 2025 has had about two months to come back, not twelve, so their twelve month columns are not twelve month figures at all.
>
> Average them in with people who have had a full year and the answer comes out too low, worse the more recent customers there are. So they are flagged rather than dropped, and every twelve month average filters on `has_full_12m`. Dropping them would lose their revenue from every other total.

Customers first acquired in a sale (people; then, on mature customers, first order value after refunds, first order gross profit, repeat rate and twelve month gross profit):

First order price group	People	First order value	First order gross profit	Repeat	Twelve month gross profit
outside any sale	36,263	$159.59	$102.47	19.3%	$126.01
sale, used the code	15,247	$124.32	$71.06	18.0%	$92.54
sale period, full price	6,591	$167.37	$106.06	24.9%	$138.97

No sale code was ever used outside a sale, so the three groups cover everyone.

RFM scores and groups

Scores cut into fifths keep ties together: each person gets the average rank of everyone sharing their value, so two people with the same value can never land in different scores. No value of R, F or M is split across two scores.

Score	R, days since last order	F, orders	M, lifetime gross profit
5	0 to 123	6 or more	$195.16 to $1,030.55
4	124 to 284	4 or 5	$115.45 to $195.15
3	285 to 469	3	$71.69 to $115.44
2	470 to 733	2	$46.68 to $71.68
1	734 to 1,095	1	-$378.16 to $46.67

R and M each put 20.0 percent of people in every score. F does not: 45,066 people (77.6 percent) have one order, which is why it uses fixed bands. The ten groups come from R and F, checked in this order, first match wins: Champions (R 4 or 5, F 4 or 5), Loyal (R 3 or more, F 3 or more), Potential loyalists (R 4 or 5, F 2), New customers (R 5, F 1), Promising (R 4, F 1), Needs attention (R 3, F 2), About to sleep (R 3, F 1), At risk (R 1 or 2, F 3 or more), Hibernating (R 1 or 2, F 2), Lost (R 1 or 2, F 1).

Lifecycle stages

The ten groups roll up into four stages, the level the CRM plan works at.

Stage	RFM groups	Days since last order	Orders	People	Opted in	Gross profit per person
New	New customers, Promising	0 to 284	1	15,851	10,162	$94.29
Active repeat	Champions, Loyal, Potential loyalists	0 to 469	2 to 7	8,025	5,120	$273.51
Cooling	Needs attention, About to sleep	285 to 469	1 or 2	10,991	7,000	$109.89
Lapsed	At risk, Hibernating, Lost	470 to 1,095	1 to 6	23,234	14,901	$108.63

In scores: New is R 4 or 5 with F 1; Active repeat is R 4 or 5 with F 2 to 5, or R 3 with F 3 to 5; Cooling is R 3 with F 1 or 2; Lapsed is R 1 or 2 with any F. A CRM audience is a filter on this table, for example `where marketing_opt_in and lifecycle_stage = 'New'`.

Retention by month since the first order

The share of people who ordered in each month after their first, counting only people who have lived through that month:

Month	0	1	2	3	4	5	6	7	8	9	10	11	12
Active %	100	0.63	1.18	1.66	1.95	2.06	2.25	2.28	2.35	2.30	2.19	2.19	2.14

Month 1 is 347 of 55,195 people and month 12 is 784 of 36,569, exactly the mature customer count. Retention climbs to a peak in month 8 rather than decaying, because people buy for a later term, not the next month. Any retention chart from this data needs that sentence next to it.

clean.unit_economics

One row per acquisition channel: what a customer is worth, what they cost and how much room each channel has. Written by `01_customer_value.ipynb`, not by SQL. 13 rows covering all 36,569 mature customers. The FY2026 budget planner reads it.

Column	Type	Meaning
channel	STRING	Channel key, as in `channel_daily`
channel_label	STRING	Display name, as on the report
family	STRING	Google, Meta, Affiliate, or Owned and organic
mature_new_customers	INT64	New customers with a full twelve months behind them
repeat_rate_12m_pct	FLOAT64	Share of them who ordered again within twelve months
gp_12m_per_customer	FLOAT64	Twelve month value: gross profit in the 365 days from the first order
first_order_gp_per_customer	FLOAT64	The part of it from the first order
ltv_before_acq_cost	FLOAT64	Twelve month value, with first order commission added back for the two affiliate channels. The value side of LTV to CAC
cost_basis	STRING	Ad spend; Commission on first order; or No cost data
cac_full_window	FLOAT64	Acquisition cost over all three years, as on the report. Blank with no cost data
cac_mature_window	FLOAT64	Acquisition cost on spend and new customers to 30 September 2024. Used for the ratio
ltv_to_cac	FLOAT64	`ltv_before_acq_cost / cac_mature_window`
payback	STRING	The payback month as text, or no cost data
breakeven_cac	FLOAT64	The highest CAC twelve months of gross profit still covers
breakeven_incrementality_pct	FLOAT64	Share of credited customers that must be genuinely new for the channel to break even over twelve months
sale_code_share_pct	FLOAT64	Share of mature new customers whose first order used a sale code
monthly_return_pct	FLOAT64	Average share of a cohort active in months 1 to 12, built on the full grid
payback_month	INT64	The payback month as a number, blank with no cost data

The seven channels with a cost:

Channel	CAC, three years	CAC, mature	LTV before acquisition cost	LTV to CAC	Break even incrementality
Google Brand	$15.59	$14.49	$131.65	9.09	11.0%
Affiliate Coupon	$11.00	$11.03	$92.99	8.43	11.9%
Affiliate Content	$16.70	$16.56	$137.96	8.33	12.0%
Google Non Brand	$34.89	$33.02	$134.66	4.08	24.5%
Google Shopping	$35.87	$36.41	$112.19	3.08	32.5%
Meta Retargeting	$39.85	$38.84	$97.26	2.50	39.9%
Meta Prospecting	$52.54	$53.08	$92.94	1.75	57.1%

All seven pay back in month 0, on the first order. Email, Organic Search, Direct, Referral and the two untagged paid buckets have no cost recorded against them, so their cost, ratio, payback and break even columns are blank. Across the whole business, ad spend per new customer is $21.47 over three years and $20.46 on the mature window, and LTV to CAC is 5.81.

WHY THE AFFILIATE ROWS ADD COMMISSION BACK

Gross profit already has affiliate commission taken out, including the commission on the first order, and that same commission is the affiliate CAC. Dividing one by the other as they stand would count it twice.

Affiliate Coupon: twelve month value $81.95, plus $11.03 of first order commission added back, gives $92.99 (rounding). Over a CAC of $11.03 that is 8.43. Paid channels need nothing added back, because ad spend never touches gross profit.

clean: the reconciliation and attribution tables

Built by `06_reconciliation_attribution.sql`. The walk from what the platforms claim down to the paid orders that happened is drawn in `02_reconciliation_attribution.ipynb` from `platform_weekly` and `order_journeys`; it is not stored as a table.

clean.platform_weekly

One row per week and ad platform for all three years: what Google or Meta claimed, next to the paid orders the order system recorded. 316 rows, 158 weeks for each platform, built on a calendar so a week with spend and no orders still has a row.

Column	Type	Meaning
week	DATE	Monday of the week
platform	STRING	google or meta
in_reconciliation_period	BOOL	One of the 51 complete weeks of the GA4 year
duplicate_event_week	BOOL	20 or 27 January or 3 February 2025, the weeks GA4 sent some purchases twice
spend_usd	FLOAT64	Ad spend
clicks	INT64	Clicks
platform_click_conversions	FLOAT64	Google's conversions, or Meta's purchases after a click
platform_view_conversions	FLOAT64	Meta's purchases after only a view; 0 for Google. Read from `raw.meta_ads`, because `clean.ad_spend` adds the two together
platform_revenue_usd	FLOAT64	Revenue the platform claims. For Meta, click purchases only
orders	INT64	Paid orders the order system credits to the platform, untagged included
revenue_before_refunds_usd	FLOAT64	Their revenue before refunds
refunded_usd	FLOAT64	Their refunds
net_revenue_usd	FLOAT64	**Net revenue, after refunds**, despite the name
gross_profit_usd	FLOAT64	Their gross profit

Platform	Spend	Paid orders	Before refunds	Net revenue	Claimed revenue	Click conversions	View conversions
Google	$609,509.99	23,622	$3,906,920.04	$3,781,607.46	$4,453,888.72	29,086.2	0
Meta	$637,778.95	14,614	$2,132,748.49	$2,075,204.02	$2,601,953.06	17,849.1	5,608.2
Both	$1,247,288.94	38,236	$6,039,668.53	$5,856,811.48	$7,055,841.78	46,935.3	5,608.2

The paid orders and revenue equal the seven paid channels in `channel_daily`. For every dollar of net revenue on its orders, Google claims $1.178, Meta $1.254 and the two together $1.205. Refunds on paid orders, which the platforms never subtract, come to $182,857 over three years. Over the 51 weeks: 15,280

paid orders, $2,364,481.82 of net revenue and $489,212.69 of spend, the same as `channel_daily`.

Claims per order are steady. In the 48 ordinary weeks of the GA4 year Google claimed 1.229 to 1.235 conversions per paid order and Meta 1.215 to 1.227 purchases after a click; none of the three duplicate weeks sat above those ranges.

clean.journey_touches

One row per session in the 30 days up to and including the session an order was bought in, for every order GA4 saw. 51,375 rows covering 27,905 orders.

Column	Type	Meaning
order_id	STRING	The order the journey ends in
person_id	STRING	The person
order_ts_utc	TIMESTAMP	When the order was placed
order_week	DATE	Monday of that week
order_channel	STRING	The channel on the order, last click in the order system
net_revenue_after_refund_usd	FLOAT64	The order's net revenue, **repeated on every row of the journey**
gross_profit_usd	FLOAT64	The order's gross profit, repeated likewise
session_key	STRING	The session, as in `clean.sessions`
user_pseudo_id	STRING	The browser
from_another_browser	BOOL	A different browser from the one they bought on, linked through a login
session_start_ts	TIMESTAMP	When the session started
days_before_order	FLOAT64	Days from session start to the order
touch_channel	STRING	The session's channel
touch_channel_group	STRING	paid, affiliate, owned or organic
is_converting_session	BOOL	The session they bought in. Exactly one per order
touch_number	INT64	Position in the journey, 1 first. The converting session is always last
touches_in_journey	INT64	Sessions in the journey
credit_last_click	FLOAT64	1 on the last session, 0 elsewhere
credit_first_click	FLOAT64	1 on the first session
credit_linear	FLOAT64	1 / touches_in_journey on every session
credit_position_based	FLOAT64	0.4 to the first and last, 0.2 shared across the middle; 0.5 each for two sessions; 1 for one

Credit is per order, so money is credit times the order's money. Never sum `net_revenue_after_refund_usd` straight from this table; multiply by a credit column first. Every rule hands out exactly one order of credit per journey, and every rule credits the same $4,442,205 of net

revenue and $2,771,238 of gross profit. 46.2 percent of journeys are one session. A login linked a second browser into the journey on 3 orders. GA4's last click agrees with `order_channel` on 97.99 percent of orders; the 562 that differ are a tagged campaign on one side and the untagged bucket of the same platform on the other.

READING A JOURNEY

Three rows for one $150 order: Meta Prospecting on the first day, Email on the fifth, and Google Brand on the ninth, the session they bought in.

`credit_last_click` is 0, 0, 1. `credit_first_click` is 1, 0, 0. `credit_linear` is 0.33 on each. `credit_position_based` is 0.4, 0.2, 0.4.

Under position based, Meta Prospecting is credited 0.4 x $150 = $60, Email $30 and Google Brand $60, and the order is still counted once across the three.

clean.order_journeys

One row per order placed from 1 October 2024 to 30 September 2025, with what GA4 saw of it. 31,692 rows. Orders GA4 missed are kept, with their journey columns empty.

Column	Type	Meaning
order_id	STRING	As in `clean.orders`
person_id	STRING	As in `clean.orders`
order_ts_utc	TIMESTAMP	As in `clean.orders`
order_date	DATE	As in `clean.orders`
order_week	DATE	As in `clean.orders`
market	STRING	As in `clean.orders`
channel	STRING	As in `clean.orders`
channel_group	STRING	As in `clean.orders`
is_brand_search	BOOL	As in `clean.orders`
ad_platform	STRING	As in `clean.orders`
revenue_before_refunds_usd	FLOAT64	Revenue before refunds
refunded_usd	FLOAT64	Refunds
net_revenue_after_refund_usd	FLOAT64	**Net revenue**
gross_profit_usd	FLOAT64	Gross profit
ga4_purchase_events	INT64	Purchase events in `raw.events` for this order, repeats included. 0 if GA4 missed it
seen_in_ga4	BOOL	GA4 recorded at least one purchase event
has_journey	BOOL	The order has a journey in `journey_touches`. Always equal to `seen_in_ga4`
touches_in_journey	INT64	Sessions in the journey
channels_in_journey	INT64	Distinct channels in it

Column	Type	Meaning
used_another_browser	BOOL	Any session on a second browser
first_touch_channel	STRING	Channel of the first session
last_touch_channel	STRING	Channel of the session they bought in
google_click_30d	BOOL	A Google session in the 30 days before the order, so Google could claim it
meta_click_7d	BOOL	A Meta session in the 7 days before, so Meta could claim it

31,692 orders, 27,905 seen by GA4 by both routes, with 28,421 purchase events (516 repeats): GA4 saw 88.1 percent of the orders. Of the 2,499 orders bought in a brand search session, 84.0 percent began on another channel.

clean.affiliate_transactions

One row per affiliate network transaction on a real order, with the order and, where GA4 saw it, the session it was bought in. 17,344 rows: the 17,369 raw transactions less the 25 on staff test orders.

Column	Type	Meaning
transaction_id	STRING	As in `raw.affiliates`
partner_id	STRING	As in `raw.affiliates`
partner_type	STRING	coupon or content
status	STRING	approved, pending or reversed
commission_usd	FLOAT64	Commission as the network reports it
commission_standing_usd	FLOAT64	Commission unless reversed. The cost
network_order_value_usd	FLOAT64	The order value the network claims against, equal to revenue before refunds
click_ts	TIMESTAMP	The partner click
order_id	STRING	As in `clean.orders`
person_id	STRING	As in `clean.orders`
order_ts_utc	TIMESTAMP	As in `clean.orders`
order_date	DATE	As in `clean.orders`
market	STRING	As in `clean.orders`
device	STRING	As in `clean.orders`
order_channel	STRING	The channel last click credits the order to
order_channel_group	STRING	Its group
credited_to	STRING	this partner type, the other partner type, or another channel
is_first_order	BOOL	The person's first order
seconds_click_to_order	INT64	Seconds from the partner click to the order

Column	Type	Meaning
click_to_order_band	STRING	1. under 1 minute, 2. 1 to 10 minutes, 3. 10 to 60 minutes, 4. 1 to 24 hours, 5. over 24 hours. The last band never occurs
revenue_before_refunds_usd	FLOAT64	From `clean.orders`
refunded_usd	FLOAT64	From `clean.orders`
net_revenue_after_refund_usd	FLOAT64	From `clean.orders`
gross_profit_usd	FLOAT64	From `clean.orders`
order_refunded	BOOL	Any refund on the order
days_order_to_refund	INT64	Days from the order to its refund
seen_in_ga4	BOOL	GA4 saw the order
session_channel	STRING	GA4 channel of the session they bought in
converting_session_start_ts	TIMESTAMP	When that session started
session_started_before_click	BOOL	The buying session had started before the partner click. Null when GA4 did not see the order

GA4 places events at fixed offsets inside a session, so no step inside the session, such as reaching checkout, is compared with the click time. The session start is, because it always falls before the order.

Measure	Coupon	Content
Transactions	14,593	2,751
Commission still standing	$181,394.58	$45,941.56
Commission rate	8.0%	10.0%
Clicks within 10 minutes of the order	69.8%	2.7%
Median click to order	435 seconds	7,380 seconds
Credited to that partner type	31.3%	100.0%
First orders	77.9%	100.0%
Average order after refunds	$153.17	$163.49

$227,336.14 still standing in all, equal to `clean.orders`; 304 transactions were reversed, $3,799.51. $128,942.95 of coupon commission, 71.1 percent of it and 56.7 percent of all commission, sits on orders last click gives to another channel. On the 3,728 of those GA4 saw, the buying session had started before the coupon click on 94.8 percent, against 10.8 percent on coupon credited orders and 11.8 on content. 530 transactions with $7,903.14 still standing are on orders later refunded, 4 to 44 days after the order.

Two commission terms are tested on this table. **Pay on credit**: coupon partners are paid only on orders the order system credits to them, which would have saved $128,942.95 over three years. **Refund hold**: commission is held for 45 days after the order and dropped if the order is refunded in that time, which in this data catches every refunded order, $7,903.14. Both together, each transaction counted once, come to $132,606.99, 58.3 percent of standing commission.

How the tables join

```
-- people and orders
clean.customer_accounts  customer_id  ->  raw.customers.customer_id   one row per account
clean.customers          person_id                                     one row per person
clean.orders             person_id    ->  customers.person_id
                         account_id   ->  customer_accounts.customer_id
clean.customer_value     person_id    ->  customers.person_id          one row per person
                         first_order_id -> orders.order_id
clean.unit_economics     channel      ->  customer_value.acquisition_channel

-- daily tables, on the same three keys
clean.channel_daily      date + channel + market
clean.funnel_daily       date + channel + market (+ device)
clean.ad_spend           report_date + channel + market
clean.sessions           session_date + channel + market

-- weekly tables
clean.campaign_weekly    week + market + channel + campaign
clean.weekly_health      week                    one row per week, GA4 year only
clean.platform_weekly    week + platform         one row per week and platform

-- reconciliation and attribution
clean.order_journeys          order_id    ->  orders.order_id          one row per order, GA4 year
clean.journey_touches         order_id    ->  order_journeys.order_id  many rows per order
                              session_key ->  sessions.session_key
clean.affiliate_transactions  order_id    ->  orders.order_id          at most one per order

-- back to raw, to show what the source said
raw.orders     order_id                 ->  clean.orders.order_id
raw.events     ecommerce.transaction_id ->  clean.orders.order_id
               user_pseudo_id           ->  clean.sessions.user_pseudo_id
```

Never join to `channel_map` on `channel`. It has several rows per channel, so a join on channel alone repeats every row once per combination. The channel is already on `orders`, `ad_spend` and `sessions`; where a list of channels is needed, take `select distinct channel`.

Daily tables. Only `channel_daily` has one row per day, channel and market. `funnel_daily` is also split by device, `ad_spend` has a row per campaign or ad set and `sessions` a row per session, so sum each of them up to day, channel and market first, or a join to `channel_daily` multiplies rows.

Weekly tables. `campaign_weekly`, `weekly_health` and `platform_weekly` are keyed on a Monday. A daily table summed to `week` lines up with them; they cannot be cut by day.

Customer tables. `customer_value` has one row per person, so it joins one to one on `person_id`. `unit_economics` has one row per channel; join it on `acquisition_channel = channel` to put channel economics next to each person.

Journey tables. `journey_touches` has one row per session, so joining it to anything at order level multiplies that thing by the journey length. Sum credit to order or channel first, or use `order_journeys`. `affiliate_transactions` holds at most one row per order, so it joins to `clean.orders` without repeating an order.

Which table answers which question

The question	The table
What did we spend and what did it return?	`channel_daily`
Which campaign inside a channel is working?	`campaign_weekly`
Can I trust this week's GA4 numbers?	`weekly_health`
Where do people drop out of the funnel?	`funnel_daily`
Which channel brings customers worth having?	`customer_value`, or `unit_economics` for the finished table
What does a customer cost, how fast do they pay back, and what is the most a channel can cost?	`unit_economics`
Who should CRM email?	`customer_value`, filtered on `lifecycle_stage`, `first_order_price_group` and `marketing_opt_in`
What does each platform claim against what happened?	`platform_weekly`, and the reconciliation notebook for the walk step by step
Who deserves the credit under a rule other than last click?	`journey_touches`
Which orders could each platform claim, and did GA4 see them?	`order_journeys`
What are coupon and content partners paid for?	`affiliate_transactions`
What did the source say before it was fixed?	the `raw` table

The metric formulas, in one place

Every ratio is built as a sum over a sum: add up the top, add up the bottom, then divide. Never average a column of ratios.

Metric	Formula	Notes
Net revenue	`sum(net_revenue_after_refund_usd)`	After discount and refunds, excluding tax and shipping charged. $11,907,875
Revenue before refunds	`sum(net_revenue_usd)` in `orders` or `channel_daily`	For tying back to the order system, always labelled. $12,296,260
Gross profit	net revenue less product cost, shipping cost, payment fee and affiliate commission	Ad spend not taken off. $7,421,060, a 62.3 percent margin
Marketing cost	ad spend plus affiliate commission	$1,474,625. Email has no cost data, so it is not in marketing cost
AOV	net revenue over orders	**After refunds.** $156.85. On revenue before refunds it would be $161.97, so AOV is always labelled after refunds
Cost per new customer	`sum(spend_usd) / sum(new_customers)` for a channel	**Blank, not zero, for a channel with no ad spend.** Zero would claim the customers were free
Paid CAC	paid ad spend over paid new customers	$37.07
Blended CAC	marketing cost over all new customers	$25.38
CTR	`sum(clicks) / sum(impressions)`	Share of ad views that were clicked
CPC	`sum(spend_usd) / sum(clicks)`	Cost per click
CPM	`1000 * sum(spend_usd) / sum(impressions)`	Cost per thousand impressions. When CPC rises and CTR holds, CPM is the cause
CPA	`sum(spend_usd) / sum(orders)`	Cost per order, on last click. Equal to cost per new customer only where every order is a first order
Internal ROAS	`sum(net_revenue_after_refund_usd) / sum(spend_usd)`	Net revenue credited by last click, over spend. 4.70 across paid channels
Platform ROAS	`sum(platform_revenue_usd) / sum(spend_usd)`	What the platforms claim. 5.66 across paid channels, always labelled platform reported
Incremental ROAS	revenue shown to be extra, over spend	**Not measured.** No channel has a holdout or experiment in the data. Break even incrementality is the line each channel must clear instead
Profit return on spend	`(sum(gross_profit_usd) - sum(spend_usd)) / sum(spend_usd)`	Gross profit left after the ads, per dollar of ads. Above 0 means the channel more than pays for its spend
MER	net revenue over marketing cost	Across all channels. 8.08

Metric	Formula	Notes
Repeat rate	mature customers who ordered again within twelve months, over mature customers	**19.6 percent**, 7,175 of 36,569
Lifetime repeat across all customers	people with more than one order ever, over all people	22.4 percent. Only quoted with this label
Twelve month value	`avg(gross_profit_12m_usd)` on mature customers	$118.82 for all mature customers
CAC by channel, for LTV to CAC	paid: ad spend over new customers; affiliates: average first order commission; everything else: blank	Two windows: three years (as on the report) and mature, to 30 September 2024, which the ratio uses. All spend is charged to acquisition, the cautious direction for brand search and retargeting
LTV before acquisition cost	twelve month value, plus first order commission for the two affiliate channels	Stops commission being counted twice
LTV to CAC	LTV before acquisition cost over mature window CAC	Whole business on ad spend per new customer, $20.46 mature: 5.81
Payback	first month where cumulative gross profit per customer passes CAC	Built on the full grid of cohort and month. All seven costed channels pay back in month 0
Break even CAC	LTV before acquisition cost	Headroom is break even CAC over CAC, less 1
Break even incrementality	CAC over LTV before acquisition cost	Meta Prospecting 57.1 percent, Google Brand 11.0
Conversion rate	GA4 orders over GA4 sessions	Built as `sum(sessions_purchased) / sum(sessions)`, the same number, because each purchasing session carries one order. 2.21 percent. **Both sides from GA4**: GA4 misses 10.5 to 12.1 percent of orders, so order system orders over GA4 sessions would overstate it
Funnel step rates	sessions reaching the next step over sessions reaching this one	View to cart 19.6, cart to checkout 56.2, checkout to purchase 29.7 percent
Capture rate	`ga4_orders / system_orders`	About 0.88 in a normal week. A sudden drop means broken tracking, not lost sales
Events per order	`purchase_events / purchased_orders`	1.00 when each purchase fires once
Claims per order	`sum(platform_click_conversions) / sum(orders)` in `platform_weekly`	Google 1.231, Meta 1.221. Steady week to week; a jump is the thing to watch
Over-claim	click conversions less repeated events less paid orders	What a platform reported beyond the orders that happened, views already out. 51 weeks: Google 2,118, Meta 1,352

Metric	Formula	Notes
Click window room	orders another platform or an unpaid channel won with this platform's click inside its window, scaled to all orders	Taken as Google 30 days, Meta 7. Share of room taken: Google 45.4 percent, Meta 44.4
Last, first, linear and position based credit	see `journey_touches`	Position based is 40, 20, 40
ROAS under another rule	last click net revenue from `channel_daily`, times the rule's revenue over last click revenue on the journeys, over spend	Assumes the orders GA4 missed split between rules like the ones it saw
Session first share	transactions whose buying session started before the partner click, over transactions GA4 saw	Read against the floor of about 11 percent on transactions the partner did start. Coupon on other channels' orders 94.8 percent
RFM scores and lifecycle stage	see `customer_value`	Recency and monetary in fifths with ties kept together, frequency on fixed bands

Traps in this data

Each of these gives a plausible wrong answer rather than an error, which is what makes them worth writing down.

1. **Averaging immature customers into twelve month value.** Filter `where has_full_12m` before averaging anything with `12m` in its name.
2. **Reading null sessions as zero traffic** before October 2024, or on 13 and 14 May 2025. The GA4 export does not reach back that far, and recorded nothing on those two days.
3. **Counting purchases from `raw.events` without removing repeats.** 517 orders were sent twice in January and February 2025. Count distinct `ecommerce.transaction_id`.
4. **Including tax in revenue.** `tax_usd` is money collected for a government.
5. **Calling the gap between `platform_conversions` and `orders` an error.** The two answer different questions. The gap is the analysis, and the reconciliation notebook sizes it step by step.
6. **Joining `funnel_daily` to `channel_daily` without summing away `device` first.** Rows multiply and every number comes out inflated.
7. **Building retention only from the months in which people ordered.** A month in which nobody from a cohort ordered has no activity rows, so reading retention from the rows that exist drops those people from the denominator as well as the numerator, and retention comes out too high. Count everyone who has lived through a month as eligible for it, active or not. The check: at month 12 the eligible count must equal the 36,569 mature customers.
8. **Taking paid totals from `campaign_weekly`.** Its spend is complete, but the 775 paid orders that lost their campaign tag ($120,928 of net revenue, 684 new customers) cannot belong to any campaign, so paid internal ROAS from it comes out at 4.60 instead of 4.70. Per tagged channel it is right. For any paid total, use `channel_daily`.

A related one: **marketing cost by channel is not ad spend by channel.** Commission sits on whichever channel last click credited, so Google Non Brand shows about $0.35M of marketing cost against $331,980 of ad spend ($20,614.53 of it coupon commission), and organic search shows marketing cost with no ad spend at all. The totals are right.

9. **Counting affiliate commission twice in LTV to CAC.** Gross profit already has it taken off, and for the affiliate channels the first order commission is also the CAC. Add it back to the value side first, as `ltv_before_acq_cost` does. It is also why the whole business ratio uses ad spend per new customer rather than blended CAC.
10. **Cutting frequency into fifths, or breaking ties by row order.** 77.6 percent of people have one order, so equal fifths are impossible. Ranking by row order would deal identical customers into different scores and spread the 45,066 one order customers across several. Use fixed bands for frequency and the average rank rule for recency and monetary.

A related caution: **order lines are not revenue.** `line_total_usd` is before refunds and a cent or two out on discounted orders. Use it for what was bought only.

11. **Reading the reconciliation tables as if they covered the same ground.** `platform_weekly` and `affiliate_transactions` cover three years, `order_journeys` the GA4 year, `journey_touches` only the orders GA4 saw (88 percent of them), and the reconciliation the 51 complete weeks. A journey total ($4,442,205) is not comparable with a year total from `channel_daily` without scaling. And `platform_weekly.net_revenue_usd` is after refunds, unlike the column of the same name in `orders`.

12. **Sizing the platforms' over-claim from the journeys.** The journeys show how many orders each platform could claim inside its click window, about twice what the platforms actually over-claim. Taking that room off the platform's report leaves a large remainder with the wrong sign. Size the over-claim from the platform's own report and use the journeys only to split it between double counting and the click window.
13. **Timing anything inside a session from GA4.** Events sit at fixed offsets from the session start, so time to checkout, time on site, or which came first between a checkout and a partner click cannot be read from this export. Use session start times only.